AD PATEL INSTITUITE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

KNAPSACK OPTIMISATION SYSTEM
IN
Design And Analysis Of Algorithm

A mini project report submitted

In partial fulfilment of the requirements for the Degree of
Bachelor of Technology

In
Information Technology
Semester – V
By
HET J PATEL(12402080601045)

Faculty Guide
Asst Prof. Minakshi Patil

Academic Year 2026-27
Department ofInformationTechnology
AD Patel Institute of Technology



AD PATEL INSTITUITE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini-project report entitled “Knapsack Optimisation System” has been prepared and submitted by Het J Patel (12402080601045) as part of the academic requirements for the course Design and Analysis of Algorithms.

This report describes a Python-based application that solves the 0/1 Knapsack Problem using Dynamic Programming, computes the maximum achievable value, and reconstructs the selected items. The work is presented for academic evaluation, subject to implementation demonstration and verification.

Guide

Minakshi Patil

(Assistant Prof.)

Head Of Department

Dr.Priyang Bhatt

(Associate Prof. & Head)

Acknowledgement

I express my sincere gratitude to the Department of Information Technology, A D Patel Institute of Technology, for providing the opportunity to undertake this mini project as part of Design and Analysis of Algorithms.

I thank my faculty guide, Asst. Prof. Minakshi Patil, for valuable guidance and encouragement. This project helped me understand optimization problems, Dynamic Programming, computational complexity, Python implementation and resource allocation.

I also acknowledge the support of my classmates, teachers and family

NAME: Het J Patel
E.NO:12402080601045

ABSTRACT

The Knapsack Optimisation System is an educational Python mini project that solves the 0/1 Knapsack Problem. Given items with individual non-negative integer weights and values and a knapsack with limited capacity, the program determines the subset with maximum total value without exceeding the capacity. Each item may be selected at most once.

The proposed solution uses Dynamic Programming to build a two-dimensional table of optimal values for progressively larger item subsets and capacities. Backtracking through this table identifies the selected items. The conventional implementation takes O(nW) time and O(nW) space for n items and integer capacity W.

The system demonstrates input validation, recurrence formulation, exact optimization, solution reconstruction and algorithm analysis. It is intended for classroom demonstrations and modest integer capacities. Python 3 is used for the implementation.

TABLE OF CONTENTS

1. Introduction 6
2. Problem Statement And Objectives 7
3. Scope and Feasibility.....8
4. Requirements And Tools.....9
5. System Design.....10
6. Algorithms Used.....11
7. Implementation Details.....12
8. Testing and Expected Results.....13
9. Complexity Analysis..... 15
10. Advantages, Limitations and Future Scope..... 16
11. Conclusion ……..... 17
12. References…….....18

1. INTRODUCTION

1.1 Background

Resource allocation often requires selecting a valuable combination of objects while obeying a fixed limit. Examples include loading a vehicle, choosing projects under a budget, and packing equipment. The 0/1 Knapsack Problem is a standard mathematical model for this decision.

1.2 Relevance to DAA

The project applies Dynamic Programming, which stores solutions to overlapping subproblems and combines them to obtain an optimal solution. It demonstrates state design, recurrence relations, complexity analysis and solution reconstruction.

1.3 Proposed Solution

The Python application accepts the number of items, each item's weight and value, and the maximum knapsack capacity. It computes the best attainable value and displays which items were chosen, their combined weight and remaining capacity.

1.4 Report Organization

The following sections explain the requirements, feasibility, architecture, algorithm, implementation, test cases, complexity, limitations and future improvements.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Given n items, each with non-negative integer weight w[i] and value v[i], and a non-negative integer capacity W, select a subset that maximizes the sum of values while ensuring that the sum of weights does not exceed W. Each item can be selected once or not at all.

2.2 Objectives

- Accept and validate item details and knapsack capacity.
- Implement the 0/1 Knapsack algorithm using Dynamic Programming in Python.
- Calculate the maximum achievable total value.
- Identify selected items through DP-table backtracking.
- Display total selected weight and remaining capacity.
- Analyze time and space complexity and test boundary cases.

2.3 Functional Requirements

- Enter item count, weights, values and capacity.
- Reject negative or invalid numeric input and unsupported problem sizes.
- Compute an optimal solution and display selected item numbers.
- Support zero-weight items with non-negative values.

2.4 Non-functional Requirements

The application should produce correct results for supported inputs, clearly report errors, use readable modular Python code, and account for memory and execution limits.

3. SCOPE AND FEASIBILITY

3.1 Scope

The project addresses the 0/1 variant with integer weights, values and capacity. It supports console input, exact DP optimization and selected-item reconstruction for manageable input sizes.

3.2 Out of Scope

- Fractional knapsack and unlimited item repetition.
- Multiple knapsacks or additional resource constraints.
- Distributed processing, permanent database storage and authentication.
- Very large capacities for which a full DP table is impractical.

3.3 Technical Feasibility

Python lists, loops and integer arithmetic are sufficient. No third-party packages are needed for the console

3.4 Operational Feasibility

Guided console prompts let students enter item details and inspect the result. Input errors are reported before optimization begins.

3.5 Economic Feasibility

Python and ordinary text editors are free to use. The project can be developed on a standard personal computer.

3.6 Assumptions

Weights, values and capacity are non-negative integers. Each item has a unique input position, and the DP table fits the available memory.

4. REQUIREMENTS AND TOOLS

4.1 Hardware Requirements

- Desktop or laptop with keyboard and display.
- Sufficient RAM for the (n + 1) by (W + 1) DP table.
- Storage for Python and the source file.

4.2 Software Requirements

- Windows, Linux or macOS.
- Python 3.x.
- Editor or IDE such as IDLE, Visual Studio Code or TextEdit.

4.3 Technologies Used

Technology	Purpose
Python 3	Implementation and console input/output
Python lists	Store item details and DP table Compute
Dynamic Programming	the maximum value
Backtracking	Recover the selected items
Console interface	Display prompts, errors and results

4.4 Data Structures

Two lists store item weights and values. A two-dimensional list dp stores the maximum attainable value for each item prefix and integer capacity. Another list stores selected item indices after backtracking.

5. SYSTEM DESIGN

5.1 System Architecture

User Input → Validation → Item Storage → DP Table Construction → Backtracking → Result → Display

5.2 Main Modules

- Input Module: reads item count, capacity, weights and values.
- Validation Module: checks integer input, numeric ranges and resource limits.
- Optimization Module: fills the DP table according to the recurrence.
- Reconstruction Module: traces selected items from the final state.
- Output Module: prints maximum value, selected items, total weight and remaining capacity.

5.3 Input and Output

Input: n, W and n pairs of (weight, value).
Output: optimal value, chosen item numbers, corresponding weights and values, total used capacity and unused capacity.

5.4 Data Flow

The program initializes a table of zeros. For each item and each capacity, it chooses the better of skipping or including the current item when inclusion is possible. The final cell yields the maximum value. Backtracking identifies the contributing items.

6. ALGORITHM USED

6.1 0/1 Knapsack Using Dynamic Programming

Let DP[i][c] be the maximum value obtainable using the first i items with capacity c. The base case is DP[0][c] = 0. For the current item with weight w and value v:

If w > c: DP[i][c] = DP[i-1][c]. Otherwise: DP[i][c] = max(DP[i-1][c], v + DP[i-1][c-w]).

6.2 Pseudocode

```
initialize DP[n+1][W+1] to zero
for i = 1 to n:
  for c = 0 to W:
        DP[i][c] = DP[i-1][c],
        if weight[i] <= c:
              DP[i][c] = max(DP[i][c],
                 value[i] + DP[i-1][c-weight[i]])

answer = DP[n][W]
c = W
for i = n down to 1:
   if DP[i][c] != DP[i-1][c]:
        select item i
        c = c - weight[i]
  reverse selected items
```

7. IMPLEMENTATION DETAILS

7.1 Python Source Code (Part 1)

```
def knapsack_optimization():
print("=" * 45)
print("       KNAPSACK OPTIMISATION SYSTEM")
print("=" * 45)
try:
    n = int(input("Enter number of items: "))
    if n < 0 or n > 1000:
        print("Invalid number of items!")
        return
    capacity = int(input("Enter knapsack capacity: "))
    if capacity < 0 or capacity > 100000:
        print("Invalid capacity!")
        return
    if (n + 1) * (capacity + 1) > 5000000:
        print("Problem size too large!")
        return
    weights, values = [], []
    for i in range(n):
        print(f"\nEnter details for Item {i + 1}")
        weight = int(input("Enter weight: "))
        value = int(input("Enter value: "))
        if weight < 0 or value < 0:
            print("Weight and value cannot be negative!")
            return
        weights.append(weight)
        values.append(value)
except ValueError:
print("Invalid input! Please enter integers only.")
return

dp=  [[0]*(capacity   +1)  for _ in range(n + 1)]
for i in range(1, n + 1):
    forwinrange(capacity + 1):
        dp[i][w]=dp[i-    1][w]
        ifweights[i-1] <= w:
            dp[i][w]=max(dp[i][w], values[i - 1] +
                dp[i-1][w   - weights[i - 1]])
```

7.2 Python Source Code (Part 2)

```
    maximum_profit=dp[n][capacity]
    selected_items=[]
    remaining=capacity
    total_weight=0
    foriinrange(n,    0, -1):
        ifdp[i][remaining] != dp[i - 1][remaining]:
            selected_items.append(i)
            total_weight += weights[i - 1]
            remaining -= weights[i - 1]
    selected_items.reverse()

    print("\n"+"="*45)
    print("           OPTIMISATION RESULT")
    print("="*45)
    print("MaximumProfit:",maximum_profit)
    print("\nSelected Items:")
    ifnotselected_items:
        print("No itemsselected.")
    else:
        print("Item\tWeight\tValue")
        foriteminselected_items:
            print(f"{item}\t{weights[item-1]}\t{values[item-1]}")
    print("TotalSelected  Items:", len(selected_items))
    print("TotalWeight:",total_weight)
    print("RemainingCapacity:", capacity - total_weight)
    print("TotalProfit:",maximum_profit)
    print("="*45)

if __name__ == "__main__":
    knapsack_optimization()
```

7.3 Execution

Save the two code parts together, in the displayed order, as knapsack.py. Run the program using python3 knapsack.py in a terminal.

8. TESTING AND EXPECTED RESULTS

The following are illustrative test cases. Actual execution should be confirmed by running the supplied program.

ID	Input / Action	Expected Result
T1	n=3, W=50; (10,60), (20,100), (30,120)	Maximum 220; items 2 and 3;
T2	n=2, W=0; (1,10), (2,20)	weight 50 Maximum 0; no
T3	n=1, W=5; (8,100)	selected items Maximum 0
T4	n=2, W=5; (0,7), (5,9)	Maximum 16; both selected
T5	Negative capacity	Input rejected
T6	Negative weight or value	Input rejected
T7	Two items tied for optimum	One valid optimal subset
T8	n=0, W=10	returned Maximum 0
T9	Input exceeds DP cell limit	Input rejected

8.1 Worked Example

For items (weight, value) = (10, 60), (20, 100), (30, 120) and capacity 50, selecting items 2 and 3 uses weight 20 + 30 = 50 and obtains value 100 + 120 = 220. Selecting all three would exceed capacity.

8.2 SAMPLE TEST OUTPUT

Illustrative Console Input

```
Enter number of items: 3
Enter knapsack capacity: 50
Enter details for Item 1
Enter weight: 10
Enter value: 60
Enter details for Item 2
Enter weight: 20
Enter value: 100
Enter details for Item 3
Enter weight: 30
Enter value: 120
```

Expected Console Output

```
==========================================

           OPTIMISATION RESULT
==========================================
Maximum Profit: 220

Selected Items:
Item    Weight Value
2       20      100
3       30      120
Total Selected Items: 2
Total Weight: 50
Remaining Capacity: 0
Total Profit: 220
==========================================
```

8.3 Additional Verification

Test zero-weight positive-value items, a capacity smaller than every weight, an empty item list, and tied optimal solutions. Compare small instances with exhaustive enumeration to validate the Dynamic Programming result. The above is an expected output, not a claim of recorded execution.

9. COMPLEXITY ANALYSIS

9.1 Time Complexity

The algorithm computes one DP entry for each of n items and W+1 capacity values, giving O(nW) time. Reconstructing selected items takes O(n) additional time.

9.2 Space Complexity

The full table contains (n+1)(W+1) entries and therefore uses O(nW) memory. The selected-item list uses O(n) additional space.

9.3 Summary

Operation	Time	Extra Space
Input and validation	O(n)	O(n) items
DP table construction	O(nW)	O(nW)
Backtracking	O(n)	O(n) indices
Overall	O(nW + n)	O(nW + n)

9.4 Interpretation

This approach is pseudo-polynomial: W is a numeric capacity, not its bit-length. Large capacity values can make the table impractical even when the number of items is modest. A value-only implementation can reduce memory to O(W) using one-dimensional DP, but reconstruction then requires additional care.

10. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

10.1 Advantages

- Finds an exact optimum for supported integer inputs.
- Demonstrates a clear recurrence and overlapping subproblems.
- Reconstructs chosen items rather than reporting only the maximum value.
- Uses standard Python without external dependencies.

10.2 Limitations

- Time and memory depend on numeric capacity W.
- The full DP table may be too large for high-capacity instances.
- Only the 0/1 single-capacity variant is supported.
- Data are not saved between runs; the current version has a console interface.

10.3 Future Scope

- Implement memory-optimized one-dimensional DP.
- Add a Tkinter graphical interface and file import/export.
- Compare exact DP with branch-and-bound and approximation methods.
- Support multiple constraints and additional knapsack variants.
- Add automated tests and performance benchmarking.

10.4 Practical Applications

The model can represent cargo selection, project portfolio planning under a fixed budget, choosing equipment for a field trip, and selecting tasks with limited resources when each option is indivisible.

11. CONCLUSION

The Knapsack Optimisation System demonstrates how Dynamic Programming solves a constrained selection problem by decomposing it into smaller overlapping subproblems. The Python implementation accepts items and a capacity, constructs a table of optimal values, and reconstructs a valid selection with the maximum achievable total value.

The project reinforces fundamental DAA concepts including problem formulation, state transitions, recurrence relations, correctness-oriented testing and time-space complexity. The worked example illustrates how selecting items 2 and 3 gives value 220 within capacity 50.

The full DP table is practical for moderate integer capacities but can consume substantial resources for large inputs. Future extensions could use memory optimization, additional optimization techniques and an improved graphical user interface. Overall, the project provides a useful academic foundation for understanding exact combinatorial optimization using Python.

12. REFERENCES

1. Cormen, Leiserson, Rivest and Stein, Introduction to Algorithms, MIT Press.
2. Kleinberg and Tardos, Algorithm Design, Pearson.
3. Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/
4. Course notes and classroom material for Design and Analysis of Algorithms.
5. Project source code: Knapsack Optimisation System (Python Dynamic Programming demonstration)